D&D LLM Campaign PRD

Project overview		
Project Description	**Multi-Agent LLM Storytelling System** AI-driven interactive storytelling engine using a **multi-agent architecture**. A Dungeon Master agent orchestrates the narrative, while autonomous character agents simulate player behavior. Persistent vector memory (FAISS/Chroma) enables cross-session continuity, and a rule layer ensures consistency. **Stack:** OpenAI / LLaMA · FAISS / Chroma · FastAPI · Streamlit or React	
Project manager	Mizpah Parrilla	
Team members	Abdulfetah Adem, Mizpah Parrilla, Spencer Kunz	
📅 **Target date**		
🟡 **Document status**	DRAFT	
Quick links		
🎨 **Designs**		

📹 **Loom demo**		
🗃️ **Work tracker**		

🎯 Objective

Multi-Agent D&D Simulation System

A functional AI system that simulates a complete Dungeons & Dragons campaign covering narrative generation, character decision-making, rule enforcement, and persistent memory across sessions.

Evaluated on: Narrative coherence · Memory consistency · System performance · User engagement

🌼 In Scope

- Multi-agent system (DM + 3–4 characters)
- Turn-based gameplay engine
- Dice roll simulation
- Character tracking system (HP, inventory, abilities)
- Persistent memory (shared + private)
- Session logging and recap generation
- Rule validation layer
- Web-based user interface

🚴 Out of Scope

- Real-time multiplayer networking
- Full D&D rulebook implementation
- Mobile application development
- Enterprise-scale deployment
- Monetization system implementation

👥 Stakeholders

- Project Team (Development & AI Design)
- Instructor / Academic Evaluator

- End Users (RPG players, AI enthusiasts, researchers)

⛱️ Business Case

Traditional D&D gameplay is limited by the availability of human Dungeon Masters and inconsistent storytelling quality. Existing AI storytelling tools lack structured memory and multi-agent coordination.

This system enables:

- Scalable AI-driven storytelling
- Consistent narrative experiences
- Autonomous multi-agent interaction

Resulting in improved accessibility, engagement, and experimentation in AI-driven gaming systems.

📊 Success metrics

Technical Success

- Multi-agent system runs successfully end-to-end
- Memory recall accuracy ≥ 90%
- System response time < 3 seconds
- Character state updates correctly

Product Success

- Average session duration ≥ 10 minutes
- Stable and coherent narrative generation
- Successful save/load functionality

Academic Success

- Research questions answered with evidence
- Complete documentation and reproducibility
- Professional presentation and report

🤔 Assumptions & Constraints

Assumptions

- LLM APIs are available and stable
- Vector database integration is feasible
- Users interact via web interface
- Generated data is sufficient for evaluation

Constraints

- Limited time (academic quarter)
- API rate limits and cost constraints
- Limited compute resources
- Prototype-level deployment only

👽 Research Questions

RQ1 — Multi-Agent Interaction Quality

Do multi-agent LLM systems produce more coherent and engaging narratives than single-agent systems?

Metrics: Narrative coherence, engagement duration

RQ2 — Memory Effectiveness

Does persistent memory improve narrative continuity across sessions?

Metrics: Recall accuracy, consistency score

RQ3 — Rule Enforcement Impact

Does a rule validation layer reduce inconsistencies and contradictions?

Metrics: Invalid action rate, consistency rate

RQ4 — Agent Role Specialization

Do specialized roles (DM vs. characters) enhance narrative diversity and decision quality?

Metrics: Action diversity, dialogue variation

RQ5 — Structured vs. Free Prompting
Does structured (JSON) prompting improve system stability over free-text prompting?
Metrics: Parsing success rate, error reduction

RQ6 — User Engagement
How does multi-agent AI gameplay affect user engagement compared to traditional or single-agent systems?
Metrics: Session duration, user feedback

RQ7 — Performance Trade-offs
What trade-offs exist between latency and narrative quality?
Metrics: Response time vs. coherence

RQ8 — Memory Architecture Comparison
How do shared vs. private memory systems impact agent behavior and story continuity?
Metrics: Consistency, agent individuality

This version is:

- **Clean and compact**
- **Consistent structure (question → metrics)**
- **Research-grade clarity (no redundancy, precise wording)**

🌟 Milestones

🗒️ Requirements

Functional Requirements

Multi-agent orchestration · Turn-based interaction engine · Dice roll simulation · Character state management · Persistent memory · Session management (save / load / logging) · Rule validation layer

Non-Functional Requirements

Category	Requirement
Performance	< 3 seconds response time
Reliability	≥ 95% stability
Scalability	Modular architecture
Security	API key protection
Usability	Intuitive UI

🔒 Deliverables

- Functional MVP system
- Multi-agent architecture implementation
- Memory system integration
- Code repository (GitHub)
- Technical documentation
- Final presentation and demo

🎨 Design

❓ Open questions

Question	Answer	Date Answered

Risks & Mitigation Strategies

Risk	level	Response / Mitigation
LLM hallucination		Structured prompts + validation
Memory inconsistency		Vector DB + schema enforcement
Performance latency		Optimization + caching
Scope creep		Strict MVP scope control

🗃️ Reference Links